

Computer Graphics (CSIT 401)

IV semester, BScCSIT

Compiled by **Ankit Bhattarai**
Ambition Guru College

Syllabus

Unit	Contents	Hours	Remarks
1.	Introduction to Computer Graphics	5	
2.	Scan Conversion Algorithms	7	<i>Very Important</i>
3.	2D Transformations and Viewing	6	<i>Very Important</i>
4.	3D Transformations and Viewing	5	
5.	3D Object Representation	6	
6.	Solid Modeling	3	
7.	Visible Surface Detection	5	
8.	Illumination, Shading and Rendering Techniques	5	
9.	OpenGL and Graphics Programming	3	

Unit 9

OpenGL and Graphics Programming

(3 hrs.)

OpenGL coordinate systems, Drawing primitives: points, lines, polygons, OpenGL transformations and viewing functions, Using modern OpenGL (shader pipeline basics).

Section 1: OpenGL, Coordinate System in OpenGL, Features

Introduction to OpenGL

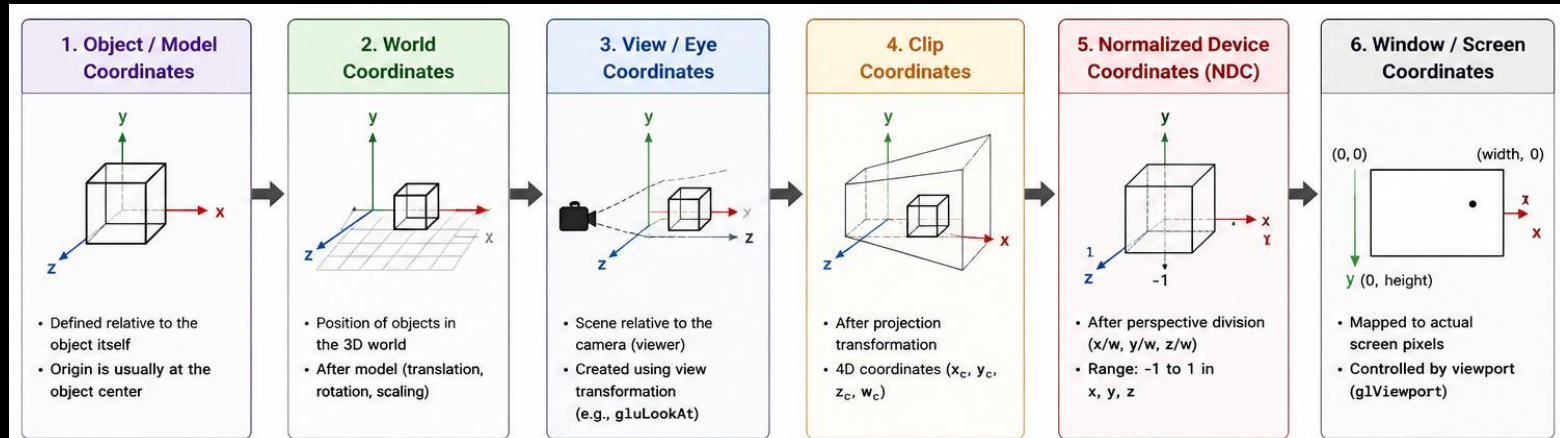
- OpenGL (Open Graphics Library) is a cross-platform, open-source graphics API (Application Programming Interface) used for rendering 2D and 3D vector graphics.
- It provides a set of functions for interacting with graphics hardware to create interactive applications, including video games, simulations, visualizations, and more.
- The API is typically used to interact with a graphics processing unit (GPU), to achieve hardware-accelerated rendering.
- Originally developed by Silicon Graphics Inc. (SGI) in the early 1990s, OpenGL has become an industry standard for graphics programming due to its versatility, performance, and widespread support across different platforms and devices.

Coordinate System in OpenGL

In OpenGL, an object passes through several coordinate systems before it finally appears as pixels on the screen. Each coordinate system has a specific purpose.

Overall Transformation Pipeline

Object Coordinates → World Coordinates → View/Eye Coordinates → Clip Coordinates → NDC → Window/Screen Coordinates



Coordinate System in OpenGL

Coordinate System

Object / Model

World

View / Eye

Clip

NDC

Window / Screen

Main Purpose

Defines the object's shape

Positions objects in the scene

Represents objects relative to the camera

Applies projection and prepares for clipping

Normalizes coordinates to a standard range

Maps coordinates to actual pixels

Introduction to OpenGL: Key Features

1. **Cross-Platform Compatibility:** OpenGL works on multiple platforms including Windows, macOS, Linux, Android, and iOS.
2. **Hardware Acceleration:** Utilizes graphics hardware to accelerate rendering, ensuring high performance.
3. **Rendering Pipeline:** Offers a programmable pipeline for controlling various stages of rendering.
4. **Support for 2D and 3D Graphics:** Capable of rendering both 2D and 3D graphics.
5. **Shader-Based Rendering:** Allows developers to write custom shaders for advanced graphics effects.
6. **Versatility and industry standard:** Suitable for a wide range of applications, from simple 2D interfaces to complex 3D simulations and games . It is widely adopted in the graphics programming community, making it an industry standard.
7. **Open-Source:** Being open-source, OpenGL fosters a collaborative development environment and allows for community contributions.
8. **Compatibility with Legacy Systems:** Maintains backward compatibility with older versions, ensuring existing codebases remain functional.

Section 2: Callback functions, Color Commands

OpenGL: Callback Functions

- In OpenGL, callback functions are functions that are registered to handle specific events or tasks within the OpenGL rendering pipeline.
- These functions are invoked by OpenGL in response to certain events or conditions, allowing developers to customize the behavior of their OpenGL applications.
- **Some of them are:**
 1. Display Callback (glutDisplayFunc)
 2. Reshape Callback (glutReshapeFunc)
 3. Keyboard Callback (glutKeyboardFunc, glutSpecialFunc)
 4. Mouse Callback (glutMouseFunc, glutMotionFunc, glutPassiveMotionFunc)
 5. Idle Callback (glutIdleFunc)

OpenGL: Callback Functions

Display Callback (glutDisplayFunc):

- This callback function is invoked whenever the contents of the window need to be redrawn.
- Developers typically use this callback to specify the rendering code for the scene.

Example:

```
void display() {  
    // Rendering code here  
    glutSwapBuffers(); // Swap front and  
    back buffers  
}  
  
glutDisplayFunc(display);
```

Reshape Callback (glutReshapeFunc):

- Invoked whenever the window is resized.
- Developers use this callback to adjust the viewport and projection matrix.

Example:

```
void reshape(int width, int height)  
{  
    glViewport(0, 0, width, height);  
    // Update projection matrix based on new  
    window dimensions  
    // ...  
}  
  
glutReshapeFunc(reshape);
```

OpenGL: Callback Functions

Keyboard Callback (`glutKeyboardFunc`, `glutSpecialFunc`):

- These callbacks are invoked when keyboard keys are pressed.
- Developers use these callbacks to handle keyboard input and perform corresponding actions.

Example:

```
void keyboard(unsigned char key, int x,
int y) {
    switch(key)
    { case
      'q':
        exit(0);
        break;
      // Handle other keys
    }
}

glutKeyboardFunc(keyboard);
```

Mouse Callback (`glutMouseFunc`, `glutMotionFunc`, `glutPassiveMotionFunc`):

- These callbacks are invoked in response to mouse events such as button clicks and movement.
- Developers use these callbacks to handle mouse input and update the scene accordingly.

Example:

```
void mouse(int button, int state, int x,
int y) {

    if (button == GLUT_LEFT_BUTTON && state ==
    GLUT_DOWN) {
        // Handle left mouse button click
    }
}

glutMouseFunc(mouse);
```

OpenGL: Callback Functions

Idle Callback (glutIdleFunc):

- Invoked when the application is idle (i.e., no events are occurring).
- Developers use this callback to perform continuous animation or other tasks.

Example:

```
void idle() {  
    // Update scene and trigger redraw  
    // ...  
}
```

```
glutIdleFunc(idle);
```

OpenGL: Color Commands

OpenGL: Color Commands

In OpenGL, color commands are used to specify the color attributes of geometric primitives such as points, lines, and polygons. These commands allow developers to control the appearance of rendered objects by setting their color properties.

Here are some common color-related commands in OpenGL:

glColor3f(r, g, b):

Sets the current color using RGB values specified as floating-point numbers between 0.0 and 1.0 for red (r), green (g), and blue (b) components.

Example:

```
glColor3f(1.0, 0.0, 0.0); // Set color to red
```

OpenGL: Color Commands

✓ `glColor4f(r, g, b, a):`

Similar to `glColor3f`, but also allows setting an alpha (transparency) value.

The alpha value ranges from 0.0 (completely transparent) to 1.0 (completely opaque).

Example:

`glColor4f(0.0, 1.0, 0.0, 0.5);` // Set color to semi-transparent green

✓ `glColor3ub(r, g, b):`

Sets the current color using RGB values specified as unsigned bytes (range 0 to 255) for red (r), green (g), and

blue (b) components.

Example:

`glColor3ub(255, 0, 0);` // Set color to red

Section 3: Drawings pixels, lines, polygons using OpenGL

Drawings pixels, lines, polygons using OpenGL

Drawing pixels using OpenGL

To draw individual pixels, you can use **glBegin(GL_POINTS)** and **glVertex2f(x, y)** functions to specify the coordinates of each pixel. Here's an example:

```
Install PyOpenGL pip install PyOpenGL  
and  
PyOpenGL_accelerate  
pip install PyOpenGL_accelerate
```

```
from OpenGL.GL import *  
from OpenGL.GLUT import *  
from OpenGL.raw.GLU import gluOrtho2D
```

```
def draw_pixel(x, y):  
    glBegin(GL_POINTS)  
    glVertex2f(x, y)  
    glEnd()
```

```
def display():  
    glClear(GL_COLOR_BUFFER_BIT)  
    glColor3f(1.0, 1.0, 1.0) # Set color to white  
  
    # Draw a pixel at (100, 100)  
    draw_pixel(100, 100)
```

```
glFlush()
```

```
def main():  
    glutInit()  
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB)  
    glutInitWindowSize(400, 400)  
    glutCreateWindow(b"Drawing Pixels")
```

```
glClearColor(0.0, 0.0, 0.0, 1.0)  
glMatrixMode(GL_PROJECTION)  
glLoadIdentity()  
gluOrtho2D(0, 400, 0, 400)
```

```
glutDisplayFunc(display)  
glutMainLoop()
```

```
if __name__ == "__main__":  
    main()
```

Drawing lines using OpenGL

To draw lines, you can use

glBegin(GL_LINES) to begin line drawing mode, and then use **glVertex2f(x1, y1)** and **glVertex2f(x2, y2)** to specify the endpoints of each line. Here's an example:

```
from OpenGL.GL import *
from OpenGL.GLUT import *
from OpenGL.raw.GLU import gluOrtho2D
```

```
def draw_line(x1, y1, x2, y2):
    glBegin(GL_LINES)
    glVertex2f(x1, y1)
    glVertex2f(x2, y2)
    glEnd()
```

```
def display():
    glClear(GL_COLOR_BUFFER_BIT)
    glColor3f(1.0, 1.0, 1.0)
    draw_line(100, 100, 300, 300)

    glFlush()
```

```
def main():
    glutInit()
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB)
    glutInitWindowSize(400, 400)
    glutCreateWindow(b"Drawing Lines")
```

```
glClearColor(0.0, 0.0, 0.0, 1.0)
glMatrixMode(GL_PROJECTION)
glLoadIdentity()
gluOrtho2D(0, 400, 0, 400)
```

```
glutDisplayFunc(display)
glutMainLoop()
```

```
if __name__ == "__main__":
    main()
```

Drawing polygons using OpenGL

To draw polygons, you can use

glBegin(GL_POLYGON) to begin polygon drawing mode, and then use **glVertex2f(x, y)** to specify the vertices of the polygon.

Here's an example:

```
from OpenGL.GL import *  
from OpenGL.GLUT import *
```

```
def draw_polygon():  
    glBegin(GL_POLYGON)  
    glVertex2f(100, 100)  
    glVertex2f(300, 100)  
    glVertex2f(200, 300)  
    glEnd()
```

```
def display():  
    glClear(GL_COLOR_BUFFER_BIT)  
    glColor3f(1.0, 1.0, 1.0) # Set color to white  
    draw_polygon()  
    glFlush()
```

```
def main():  
    glutInit()  
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB)  
    glutInitWindowSize(400, 400)  
    glutCreateWindow(b"Drawing Polygons")  
    glClearColor(0.0, 0.0, 0.0, 1.0)  
    glMatrixMode(GL_PROJECTION)  
    glLoadIdentity()  
    gluOrtho2D(0, 400, 0, 400)  
    glutDisplayFunc(display)  
    glutMainLoop()
```

```
if __name__ == "__main__":  
    main()
```

Section 4: Viewing and Lighting, Modern OpenGL Shader Pipeline

Viewing and Lighting

Viewing

In OpenGL, viewing involves setting up a virtual camera to define the perspective or orthographic projection through which the scene is viewed. Here are the main steps involved in setting up the viewing transformation:

1. Define the Camera Position and Orientation:

- This involves setting up the position and orientation of the virtual camera or eye in the scene.
- Typically done using functions like **gluLookAt()** or by manually manipulating the model-view matrix.

2. Set Up the Projection:

- Choose between perspective or orthographic projection. Perspective projection creates a realistic sense of depth, while orthographic projection maintains the same scale regardless of depth.
- Use functions like **glFrustum()** or **gluPerspective()** for perspective projection, and **glOrtho()** for orthographic projection.

3. Transformations:

- Apply any necessary transformations to the objects in the scene.
- This can include translations, rotations, and scaling operations.

Lighting

OpenGL supports various lighting techniques to simulate the interaction of light with objects in a scene. Lighting can dramatically enhance the realism and visual appeal of rendered scenes. Here are the key components of lighting in OpenGL:

1. **Light Sources:** OpenGL supports different types of light sources, such as directional lights, point lights, and spotlights. Each light source has properties like position, direction, color, and intensity.
2. **Material Properties:** Objects in a scene have material properties that determine how they reflect or absorb light. Common material properties include ambient, diffuse, specular, and emissive colors, as well as shininess.
3. **Lighting Equations:** OpenGL uses lighting equations to calculate the final color of each pixel in the scene. These equations take into account the positions and properties of light sources, material properties of objects, and the viewing direction.

Lighting

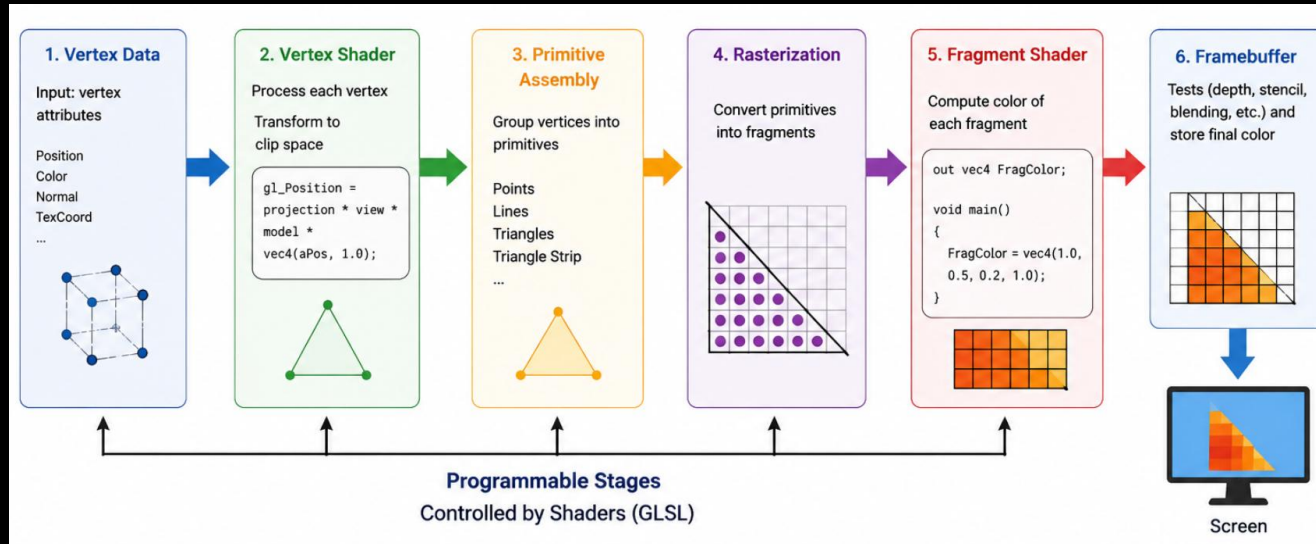
4. Shading Models: OpenGL supports different shading models to determine how light interacts with surfaces. Common shading models include flat shading, Gouraud shading, and Phong shading.

5. Enabling and Disabling Lighting: Use `glEnable(GL_LIGHTING)` to enable lighting and `glDisable(GL_LIGHTING)` to disable it. Individual lights can be enabled or disabled using `glEnable(GL_LIGHTi)` and `glDisable(GL_LIGHTi)`.

6. Lighting Model Setup: Set up the properties of light sources and material properties using functions like `glLightfv()` and `glMaterialfv()`.

Modern OpenGL Shader Pipeline

Modern OpenGL uses a **programmable rendering pipeline**, where programmers write small programs called **shaders** to control how vertices and pixels are processed. Unlike the old fixed-function pipeline, modern OpenGL mainly uses **GLSL (OpenGL Shading Language)** to control the rendering process.



Modern OpenGL Shader Pipeline

Stage	Main Function
Vertex Data	Provides vertex attributes
Vertex Shader	Processes and transforms each vertex
Primitive Assembly	Groups vertices into points, lines, triangles, etc.
Rasterization	Converts primitives into fragments
Fragment Shader	Determines fragment color and other properties
Framebuffer	Stores final rendered pixel data

Questions

1. Define OpenGL. What are its features ? List down some of the callback functions used in OpenGL.
2. Explain any two functions in OpenGL. Write the basic commands to draw the pixel and polygon in OpenGL.
3. Demonstrate how a polygon can be created using OpenGL.
4. Write short notes on OpenGL.
5. Explain modern OpenGL shader pipeline in brief.
6. Write a program to draw a line using OpenGL.

THANK YOU
Any Queries ?